

INTERNAL ENGINEERING REPORT · COMPLETE

YatruSathi — DevOps

Project start to present · every DevOps file, what it does, and why

A full account of the DevOps in the `YatruSathi-with-devops` monorepo: how it began (three separate apps on ad-hoc hosting), every change since — folder by folder, file by file — and the current state. Written so any engineer can read a section and explain it to someone else.

Repository: `github.com/kritiup/YatruSathi-with-devops` · default branch `main`

Services: Django REST API · React + Vite SPA · Flask + Socket.IO AI chatbot

Generated 2026-09-07

Contents

Part I — The story

1. Where the project started
2. Timeline, mapped to git commits

Part II — Foundations

3. Monorepo consolidation
4. Environment-variable strategy
5. Containerisation — the three `Dockerfile`s
6. Local orchestration — `docker-compose.yml`

Part III — Automation (`.github/`)

7. The CI pipeline and how it evolved
8. Dependabot, CodeQL, secret & Dockerfile scanning

Part IV — Cloud target design

9. Application changes that make the code deployable
10. Infrastructure as Code — `infra/terraform/`
11. Kubernetes manifests — `k8s/`
12. The deploy pipeline (`deploy-eks.sh` , staging, production)

Part V — Reference

13. Legacy / superseded deploy configs
14. Current status & the AWS-account constraint

How to read this

File contents appear in grey boxes; a plain-language explanation follows. Blue **Why** notes explain a design decision — memorise those for a walkthrough. Orange boxes are caveats. Green boxes are "done and verified".

1 · Where the project started

Before any DevOps work, YatruSathi was **three unrelated codebases**, each deployed by hand to a different free host:

Service	How it ran	Artefacts left behind
Django backend	Pushed to Render ; Render ran <code>scripts/build.sh</code> then <code>scripts/start.sh</code>	<code>render.yaml</code> , <code>Procfile</code> , <code>runtime.txt</code> , <code>scripts/*.sh</code>
React frontend	A GitHub Action built it and pushed <code>dist/</code> to Cloudflare Pages	– <code>yatruSathiFrontend-/.github/workflows/deploy.yml</code>
Flask chatbot	Run manually with <code>python app.py</code> — no deployment at all	none

Problems this created

- **No reproducibility** — "works on my machine"; Python/Node versions and system packages differed per developer and per host.
- **Three pipelines, three dashboards** — a change spanning frontend + backend could not be tested or shipped together.
- **Secrets in the code** — an OpenAI key, Supabase keys, a Postgres password (`YatruSathi@123` / `Minorproject@123`) and a Gmail app password were committed and pushed to public repos (see `scripts/deploy_setup.sh` , which still prints them).

- **Data model unclear** — backend had drifted from Supabase Postgres to a local SQLite file; Render's disk is wiped on every deploy, so data did not survive.
- **The chatbot was invisible** to CI, to ops, to everyone.

Two large planning documents (`AUDIT_REPORT.md` , `REFACTORING_QUICK_START.md`) were written to catalogue this and were later removed once the work they described was done.

2 · Timeline, mapped to git commits

#	Commit	Date	What changed (DevOps)
1	91a8276 / 0cc8cb2	Aug 30–31	"first commit" — the three apps pulled in as git submodule pointers (gitlinks) + the audit documents.
2	e5543be	Aug 31	Root <code>.env.example</code> , root <code>.gitignore</code> , docs/environment.md — the environment-variable inventory ("Phase 1").
3	b5dae18	Aug 31	First <code>docker-compose.yml</code> (backend + frontend) and first <code>.github/workflows/ci.yml</code> (jobs: frontend , backend , docker). Audit docs deleted.
4	da1d5ec	Aug 31	CI: bump <code>checkout@v4→v5</code> , <code>setup-node@v4→v5</code> , <code>setup-python@v5→v6</code> ; Node 20→22.
5	90ae7f6	Aug 31	Vendor the three apps — replace the submodule gitlinks with real file trees (CI was checking out empty folders and failing). This is when the pre-existing per-service <code>Dockerfile</code> s, <code>nginx.conf</code> , <code>.dockerignore</code> s and <code>.env.example</code> s entered the monorepo.
6	b098ff3 → d2eff3a	Aug 31	Dependabot (configured in the GitHub UI) opens PR #1 — 7 backend pip bumps; merged.
7	4c1c051	Aug 31	Empty commit to re-run CI on the fixed tree.
8	483abf4	Aug 31	The <code>docker</code> CI job becomes <code>security:Trivy</code> scans dependency lockfiles (reported) and the built images (fails on fixable HIGH/CRITICAL).
9	732127c	Sep 7	Big one: chatbot <code>Dockerfile</code> + deps; backend <code>DATABASE_URL</code> support; CI gains an ECR push job (OIDC) and a deploy job; full <code>infra/terraform/</code> (VPC, EKS, RDS, ECR, IAM, Secrets, add-ons); <code>k8s/</code> base + production overlay; docs/deployment-eks.md .
10	uncommitted branch	Sep 7	Supply-chain hardening (<code>dependabot.yml</code> , <code>codeql.yml</code> , gitleaks, hadolint, CI caching restored); Sentry in all three services; chatbot Redis -ready; metrics-server + Container Insights ; staging overlay + <code>deploy-staging</code> / <code>deploy-production</code> split; docs/secret-rotation.md ; this report.

The five capabilities added, in order

1. **Reproducible** — a `Dockerfile` per service (foundation for all the rest).
2. **Runnable together** — `docker-compose.yml` .
3. **Cloud-ready code** — Postgres, error tracking, scalable chatbot.
4. **Automated** — GitHub Actions build/test/scan/ship; Dependabot; CodeQL.
5. **Operable** — Terraform + Kubernetes with health checks, autoscaling, managed secrets, observability, and separate staging/production.

3 · Monorepo consolidation

The three apps live in one repository at:

Directory	Service	Stack
-yatrubackend-/	REST API & data model	Django 6, DRF, SQLite→Postgres
-yatruSathiFrontend-/	Web client	React 19, TypeScript, Vite 7, MUI 6
-Yatrusathi-AIchatbot/	AI assistant & realtime chat	Flask, Flask-SocketIO, Groq

(The odd leading/trailing dashes are the original repo names, kept so history and external links still resolve.)

The submodule → vendor fix (commit 90ae7f6)

The first commits added the three directories as **git submodule pointers** — a submodule stores only a commit hash, not the files, and needs a `.gitmodules` file plus a recursive checkout to populate. There was no `.gitmodules` , so:

```
git clone ...           → three EMPTY directories
CI: working-directory: ./-yatrubackend- → "no such file" → every job fails
```

Commit 90ae7f6 deleted the gitlinks and committed the actual file trees ("vendoring"). From then on a plain `git clone` or CI checkout has everything.

Why one repo, not three. A single pipeline can build, test and release all three services from one commit; a change that touches the API and the client is one reviewable pull request; there is one place for `docker-compose.yml` , one CI config, one Terraform root. The cost — everyone clones all three services — is negligible at this size.

4 · Environment-variable strategy

Rule: **nothing is hard-coded; every deployment-specific value comes from an environment variable; no real secret is ever committed.** Established in commit e5543be and documented in `docs/environment.md` .

The four .env.example files

File	Committed?	Purpose
.env.example	yes	Root — values <code>docker-compose.yml</code> interpolates (ports, image tag) plus per-service overrides.
-yatrubackend-/.env.example	yes	Every key Django reads: <code>SECRET_KEY</code> , <code>DEBUG</code> , <code>ALLOWED_HOSTS</code> , <code>CORS/CSRF</code> , admin account, e-mail.
-yatruSathiFrontend-/.env.example	yes	The <code>VITE_*</code> build-time values (API URL, chatbot URL, Supabase).
-Yatrusathi-AIchatbot/.env.example	yes	<code>SECRET_KEY</code> , <code>GROQ_API_KEY</code> , <code>GROQ_MODEL</code> , optional Supabase.
<each>/. <code>env</code>	no — git-ignored	The real local values. Copied from <code>.env.example</code> .

Each app already loads its own file: Django and Flask via `python-dotenv ( load_dotenv() )`, the frontend via Vite (`import.meta.env.VITE_*` , frozen in at build time).

Why .env.example is committed but .env is not. The example is the contract — the canonical list of every variable the service understands, with safe placeholder values, so a new developer knows exactly what to set. The real `.env` holds actual secrets and is git-ignored. This pattern is repeated at every layer: Terraform has `terraform.tfvars.example` , the deploy has GitHub repository variables.

`.gitignore` — the secret-safety net

```
# — Secrets / environment —
.env
.env.*
!.env.example      # ...but DO keep the template
*.pem
*.key
# — Terraform (added Sep 7) —
**/.terraform/     # provider cache – large, machine-specific
*.tfstate           # STATE FILES CONTAIN SECRETS IN PLAINTEXT
*.tfstate.*
*.tfvars            # your real variable values
!*.tfvars.example
infra/terraform/backend.tf # holds your state-bucket name
# — plus: __pycache__, node_modules, db.sqlite3, media/, dist/, *.log ...
```

The `!*.tfvars.example` / `!.env.example` lines are re-includes — ignore the whole class, keep the one template. The Terraform lock file `.terraform.lock.hcl` is committed (it pins provider checksums for reproducible installs).

`docs/environment.md`

A reference table of every variable each service reads — required vs optional, default, and notes — plus a "rotate before production" list of the credentials that had been committed. That list is now actioned by `docs/secret-rotation.md` and enforced by the `gitleaks` CI job (§8).

5 · Containerisation — the three `Dockerfile`s

A container **image** is a frozen, self-contained filesystem with the app and exactly the runtime it needs. The identical image runs on a laptop, in CI, and in the cloud. All three share two patterns:

- **Multi-stage build** — a "builder" stage installs dependencies; a slim "runtime" stage copies only the result. Build tools never ship.
- **Non-root user** — the process runs as an unprivileged `app` user.

The **backend** and **frontend** Dockerfiles pre-existed (entered the repo with the vendoring in `90ae7f6`); the **chatbot** one and its support files were written in `732127c`.

5.1 · Backend — -yatrubackend-/Dockerfile

```
FROM python:3.13-slim-bookworm AS builder
ENV PIP_NO_CACHE_DIR=1 PIP_DISABLE_PIP_VERSION_CHECK=1
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
WORKDIR /app
COPY requirements.txt .
COPY requirements/ requirements/
RUN pip install --upgrade pip && pip install -r requirements.txt

FROM python:3.13-slim-bookworm AS runtime
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 PIP_NO_CACHE_DIR=1 \
    DJANGO_SETTINGS_MODULE=backend.settings PATH="/opt/venv/bin:$PATH"
RUN groupadd --system app && useradd --system --gid app --home-dir /app app
WORKDIR /app
COPY --from=builder /opt/venv /opt/venv
COPY . .
RUN chmod +x docker-entrypoint.sh scripts/*.sh \
    && mkdir -p data media logs staticfiles \
    && SECRET_KEY=build-only DEBUG=False python manage.py collectstatic --no-input \
    && chown -R app:app /app
USER app
EXPOSE 8000
HEALTHCHECK ... CMD python -c "... GET /api/ ... status < 500 ..."
ENTRYPOINT ["/docker-entrypoint.sh"]
```

Line	What & why
FROM python:3.13-slim-bookworm AS builder	Official Python 3.13, the "slim" Debian variant (no compilers/docs). Named so the runtime stage can copy from it.
RUN python -m venv /opt/venv + ENV PATH	An isolated, portable virtual-environment. Putting its <code>bin</code> first on <code>PATH</code> makes every later <code>pip</code> / <code>python</code> use it.
COPY requirements*.txt before COPY . .	Docker layer caching: the slow <code>pip install</code> layer is reused for every build where requirements are unchanged, even when app code changes.
PYTHONDONTWRITEBYTECODE / PYTHONUNBUFFERED	No <code>.pyc</code> files; flush logs immediately.
groupadd ... useradd ... app + later USER app	Nothing runs as root after this point.
COPY --from=builder /opt/venv /opt/venv	Bring the finished dependencies without pip/build machinery.
collectstatic ... SECRET_KEY=build-only	Bake Django's admin CSS/JS into the image at build time (throw-away key just to let settings import), instead of gathering them on every start.
mkdir -p data media logs staticfiles + chown	Pre-create the volume mount-points owned by <code>app</code> , so a fresh Docker volume comes up writable, not root-owned.
HEALTHCHECK ... status < 500	Container is "healthy" if Django answers anything below 500. Inline Python because the slim image has no <code>curl</code> / <code>wget</code> . A forwarded-proto header stops the HTTPS-redirect turning it into a loop.
ENTRYPOINT ["/docker- entrypoint.sh"]	The container runs a start-up script, not the server directly.

`-yatrubackend-/docker-entrypoint.sh`

```
set -euo pipefail                                # abort on any error / unset var / failed pipe
python manage.py migrate --no-input                # apply DB schema changes
python scripts/seed_db.py || echo "...continuing" # optional sample data, non-fatal
exec gunicorn backend.wsgi:application \
  --bind "0.0.0.0:${PORT:-8000}" \
  --workers "${WEB_CONCURRENCY:-3}" \
  --timeout "${WEB_TIMEOUT:-120}" \
  --access-logfile - --error-logfile -
```

Migrations run first so the schema is always current before traffic. Gunicorn is the production server (never Flask/Django dev servers in production); `exec` makes it PID 1 so it receives stop signals. 3 sync workers by default (the backend has no long-lived connections).

5.2 · Frontend — `-yatsuSathiFrontend-/Dockerfile`

Pre-existing; **this work added the two `VITE_SENTRY_*` build arguments.**

```
FROM node:22-alpine AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci                                # install EXACTLY the locked tree
ARG VITE_API_BASE_URL
ARG VITE_CHATBOT_URL
ARG VITE_SUPABASE_URL
ARG VITE_SUPABASE_ANON_KEY
ARG VITE_SENTRY_DSN                        # ← added
ARG VITE_SENTRY_ENVIRONMENT               # ← added
ENV VITE_API_BASE_URL=$VITE_API_BASE_URL ... VITE_SENTRY_DSN=$VITE_SENTRY_DSN ...
COPY . .
RUN npm run build                          # → static files in /app/dist

FROM nginx:1.27-alpine AS serve
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
HEALTHCHECK ... wget -q -O /dev/null http://127.0.0.1/ ...
```

Why the frontend needs build *arguments*, not runtime environment variables. Vite replaces every

`import.meta.env.VITE_*` with a literal string *when the bundle compiles*. By the time nginx serves the files there is no process to read env vars — the values are frozen into the JavaScript. So the API URL, chatbot URL and Sentry DSN must be given at `docker build` time. This is exactly why the deploy script (§12) rebuilds the frontend image per environment.

`-yatsuSathiFrontend-/nginx.conf`

- `location /` → `try_files $uri $uri/ /index.html` — unknown paths return the app shell so React Router handles them (this is what makes a SPA survive a refresh / deep link).
- `location /assets/` → `Cache-Control ... immutable`, one year — build assets have content-hashed names.
- `location = /index.html` → `no-cache` — a new deploy is picked up immediately.
- `gzip` on text types; security headers (`X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`) on every response.

5.3 · Chatbot — -Yatrusathi-AIchatbot/Dockerfile (new)

```
FROM python:3.13-slim-bookworm AS builder
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
COPY requirements.txt .
RUN pip install --upgrade pip && pip install -r requirements.txt

FROM python:3.13-slim-bookworm AS runtime
ENV ... PATH="/opt/venv/bin:$PATH" GUNICORN_THREADS=50
RUN groupadd --system app && useradd --system --gid app --home-dir /app app
COPY --from=builder /opt/venv /opt/venv
COPY . .
RUN chown -R app:app /app
USER app
EXPOSE 5005
HEALTHCHECK ... CMD python -c "... GET /health ... status < 400 ..."
CMD ["sh", "-c", "exec gunicorn app:app --bind 0.0.0.0:5005 \
    --worker-class gthread --workers 1 --threads ${GUNICORN_THREADS:-50} \
    --timeout ${GUNICORN_TIMEOUT:-120} --access-logfile - --error-logfile -"]
```

Why `--workers 1`. The chatbot uses Flask-SocketIO in `threading` mode, which keeps every chat room and each user's conversation history in *one process's memory*. A second worker would have separate memory, so users would randomly get amnesiac replies. One worker with many *threads* gives concurrency without splitting that state. Scaling past one process needs Redis — §9.4 and §11.6.

Why `gthread`. Gunicorn's default "sync" worker serves one request at a time and drops idle connections — fatal for Socket.IO / Server-Sent-Events streams. `gthread` holds long-lived connections open.

Support files (new)

- `requirements.txt` — pinned versions (`Flask==3.1.3` , `Flask-SocketIO==5.6.1` , `groq` , `gunicorn` , plus `sentry-sdk` and `redis` added in §9). Before this the service was installed with an ad-hoc `pip install flask ...` that could resolve to different versions per day.
- `.dockerignore` — keeps `.git` , `.env` , `venv/` , caches out of the build context and image.
- `.flake8` — the bundled `knowledge_base.py` is a huge data-style file; CI hard-fails only on real defects and reports the rest.

The backend and frontend also carry pre-existing `.dockerignore` files with the same intent (exclude VCS, secrets, local data, docs; the backend one also excludes `render.yaml` / `Procfile` so those legacy files don't reach the image).

6 · Local orchestration — docker-compose.yml

One command — `docker compose up --build` — starts the whole stack: three services, a private network so they resolve each other by name, and named volumes so data survives a restart. First added in `b5dae18` (backend + frontend); the **entire chatbot service** and the `SENTRY_*` / `REDIS_URL` wiring were added later.

Anatomy of a service

```
backend:
  build: { context: ./-yatrubackend- }          # build the image from that folder
  image: yatrusathi-backend:${IMAGE_TAG:-local}
  restart: unless-stopped                       # auto-restart on crash
  environment:
    SECRET_KEY: ${DJANGO_SECRET_KEY:-django-insecure-...}
    DEBUG: ${DJANGO_DEBUG:-True}
    SQLITE_PATH: /app/data/db.sqlite3
    SENTRY_DSN: ${SENTRY_DSN_BACKEND:-}          # blank = disabled
    SENTRY_ENVIRONMENT: ${SENTRY_ENVIRONMENT:-development}
    ...
  ports: ["${BACKEND_PORT:-8000}:8000"]
  volumes:
    - backend_data:/app/data                    # the SQLite file
    - backend_media:/app/media                  # user uploads
  networks: [yatrusathi]
  healthcheck: { test: [python -c "... /api/ ..."], interval: 30s, retries: 3 }
```

Pattern	Meaning
<code>\${VAR:-default}</code>	Use <code>VAR</code> ; if unset <i>or empty</i> , use <code>default</code> . This is why the stack runs with no <code>.env</code> file at all .
<code>ports: ["8000:8000"]</code>	<code>host:container</code> . Left = reachable from your machine; right = inside the container.
<code>volumes: [name:/path]</code>	A Docker-managed disk. Survives <code>down</code> ; only <code>down -v</code> deletes it.
<code>networks: [yatrusathi]</code>	All three join one bridge network — <code>backend</code> can reach <code>chatbot</code> by that hostname.
<code>healthcheck</code>	Compose tracks healthy/unhealthy; other services can wait for "healthy".

The frontend service — depends_on with health gates

```
frontend:
  build:
    context: ./-yatruSathiFrontend-
    args:
      VITE_API_BASE_URL: ${VITE_API_BASE_URL:-http://localhost:${BACKEND_PORT:-8000}/api/}
      VITE_CHATBOT_URL:  ${VITE_CHATBOT_URL:-http://localhost:${CHATBOT_PORT:-5005}}
      VITE_SENTRY_DSN:   ${VITE_SENTRY_DSN:-}
  ports: ["${FRONTEND_PORT:-8080}:80"]
  depends_on:
    backend: { condition: service_healthy }
    chatbot: { condition: service_healthy }
```

Why the frontend build args say `localhost` , not `backend` . These URLs are used by the visitor's *browser*, which is outside Docker and cannot resolve the compose hostname `backend` — only the port published on the host. Container-to-container calls use the compose name; the browser cannot.

Bottom of the file: `networks: { yatrusathi: { driver: bridge } }` and `volumes: { backend_data:, backend_media: }` — declare the shared network and the two persistent disks.

7 · The CI pipeline and how it evolved

CI = every push is automatically built, tested and scanned. GitHub Actions runs it; each *job* is an isolated Linux VM. The file is `.github/workflows/ci.yml`.

7.1 · Version 1 (commit `b5dae18`) — three jobs

```
on: { push:, pull_request:, workflow_dispatch: }
permissions: { contents: read }          # least privilege – read the repo, nothing else
jobs:
  frontend: checkout → setup-node → npm ci → lint → typecheck → test → build
  backend:  checkout → setup-python → pip install → manage.py check
           → makemigrations --check → flake8 → pytest
  docker:  needs: [frontend, backend]
           checkout → docker compose build      # build both images, push nothing
```

Then commit `da1d5ec` bumped the action versions (`checkout v4→v5` , `setup-node v4→v5` , `setup-python v5→v6`) and Node 20→22 to match the Dockerfiles.

7.2 · The `security` job (commit `483abf4`)

The `docker` job was renamed `security` and given **Trivy** scanning:

```
security:
  needs: [frontend, backend]      (later: + chatbot)
  env: { TRIVY_DB_REPOSITORY: public.ecr.aws/aquasecurity/trivy-db:2 }
  steps:
    - Trivy – dependency scan  (scan-type: fs, exit-code "0")  # REPORT only
    - docker compose build --pull                                # build all images
    - Trivy – backend image    (exit-code "1", ignore-unfixed) # FAIL on fixable HIGH/CRITICAL
    - Trivy – frontend image   (exit-code "1")
    - Trivy – chatbot image     (exit-code "1")  (added later)
```

Setting	Meaning
dependency scan exit-code "0"	Never fails the build — dependency CVEs are Dependabot's job.
image scans exit-code "1" + ignore-unfixed: true	Fails the build on a HIGH/CRITICAL <i>that has a fix available</i> . Unfixable ones are ignored (nothing to do).
TRIVY_DB_REPOSITORY: public.ecr.aws/...	Pull Trivy's CVE database from AWS's mirror to dodge Docker Hub rate limits.
if: \${{ !cancelled() }} on later scans	Scan the frontend/chatbot even if the backend scan already failed — see every problem in one run.

7.3 · Current job graph

```
frontend ─┐
backend  ─┴─> security → push-ecr → deploy-staging → deploy-production
chatbot  ─┘                                     (manual only)
gitleaks  (independent)      codeql.yml runs as its own workflow
hadolint  (independent)
```

Step	Purpose
setup-node / setup-python with cache: (re-added)	Install the toolchain; cache the dependency download between runs to save minutes.
npm ci / pip install -r ...	Install the exact locked dependencies.
npm audit --audit-level=high true (added)	Report vulnerable npm packages; informational (true).
lint / typecheck / test / build	Style, TypeScript compile, unit tests, production build.
backend: manage.py check, makemigrations --check --dry-run	Django sanity; fail if a model change has no migration.
chatbot: flake8 --select=E9,F63,F7,F82	Hard-fail only on real bugs (syntax, undefined names); full style reported separately.
chatbot: python -c "import app, chatbot, knowledge_base"	Smoke test — the modules at least import.

push-ecr job — publish images

```
needs: [security]
if: github.ref == 'refs/heads/main' && (push || workflow_dispatch)
permissions: { contents: read, id-token: write } # id-token: request an OIDC token
steps:
  - docker compose build
  - aws-actions/configure-aws-credentials@v4 with role-to-assume: ${vars.AWS_DEPLOY_ROLE_ARN}
  - aws-actions/amazon-ecr-login@v2
  - for svc in frontend backend chatbot:
      docker tag yatrusathi-$svc:local $REGISTRY/yatru-sathi-$svc:$SHA
      docker tag ... $REGISTRY/yatru-sathi-$svc:latest
      docker push both
```

Why OIDC, not an access key. GitHub mints a short-lived, signed token proving "workflow X, repo Y, branch main". AWS trusts that token and returns temporary credentials (~1 hour). There is **no long-lived AWS_ACCESS_KEY_ID stored in GitHub** to leak, and the trust is scoped to exactly this repo + branch.

Why tag with the commit SHA. :latest moves; :<sha> is immutable, so a rollback can point at the exact image a given commit produced.

8 · Dependabot, CodeQL, secret & Dockerfile scanning

8.1 · Dependabot

Initially enabled in the **GitHub UI** (it opened PR #1 — b098ff3 — bumping 7 backend pip packages). Now committed as `.github/dependabot.yml` covering **five** ecosystems:

```

version: 2
updates:
  - { package-ecosystem: pip,    directory: /-yatrubackend-/requirements, groups: {backend-python: {patterns:
["*"]}} }
  - { package-ecosystem: pip,    directory: /-Yatrusathi-AIchatbot,      groups: {chatbot-python: {patterns:
["*"]}} }
  - { package-ecosystem: npm,    directory: /-yatruSathiFrontend-,
    groups: {frontend-npm: {patterns: ["*"], update-types: ["minor","patch"]}} }
  - { package-ecosystem: github-actions, directory: / }
  - { package-ecosystem: docker, directories: [/-yatrubackend-, /-yatruSathiFrontend-, /-Yatrusathi-AIchatbot] }

```

`groups` bundles routine updates into one PR per ecosystem per week. The npm group is capped at `minor / patch` so a major React bump arrives as its own reviewable PR.

8.2 · `.github/workflows/codeql.yml` (added)

```

on: { push: {branches:[main]}, pull_request: {branches:[main]}, schedule: [{cron: "27 3 * * 1"}] }
jobs:
  analyze:
    permissions: { contents: read, security-events: write }
    strategy: { matrix: { language: [python, javascript-typescript] } }
    steps: init (queries: security-and-quality) → autobuild → analyze

```

CodeQL is GitHub's deep static-analysis engine: it builds a queryable database of the code and runs security queries (injection, path traversal, unsafe deserialisation, hard-coded credentials...). Findings land in **Security** → **Code scanning**. The weekly `schedule` re-scans with new queries even when nobody pushed.

8.3 · `gitleaks` job (added)

```

gitleaks:
  steps:
    - actions/checkout@v5 with: { fetch-depth: 0 } # full history
    - gitleaks/gitleaks-action@v2

```

Scans the diff and history for anything shaped like a credential and **fails the build**. This is the guardrail against re-introducing the keys that were previously committed.

8.4 · `hadolint` job (added)

```

hadolint:
  strategy:
    matrix: { dockerfile: [ ./-yatrubackend-/Dockerfile, ./-yatruSathiFrontend-/Dockerfile, ./-Yatrusathi-
AIchatbot/Dockerfile ] }
  steps:
    - hadolint/hadolint-action@v3.1.0
      with: { dockerfile: ${ matrix.dockerfile }, ignore: DL3008,DL3013,DL3018, failure-threshold: warning }

```

A `matrix` runs the job once per Dockerfile in parallel. `hadolint` is a Dockerfile best-practice linter; the `ignore` list drops rules that don't apply to these base images.

Net supply-chain posture. Every push: dependency CVE report (Trivy fs), image CVE gate (Trivy image), secret scan (gitleaks), Dockerfile lint (hadolint), deep static analysis (CodeQL), and known-vuln npm report (npm audit). Weekly: Dependabot PRs across five ecosystems + a scheduled CodeQL re-scan.

9 · Application changes that make the code deployable

Four small, guarded source changes — behaviour is **unchanged** unless you opt in with an environment variable.

9.1 · Backend — Postgres support `-yatrubackend-/backend/settings.py`

```
DATABASE_URL = os.getenv("DATABASE_URL", "").strip()
if DATABASE_URL:
    import dj_database_url
    DATABASES = { "default": dj_database_url.parse(
        DATABASE_URL, conn_max_age=int(os.getenv("DB_CONN_MAX_AGE", "600")),
        conn_health_checks=True, ssl_require=env_bool("DB_SSL_REQUIRE", False)) }
else:
    DATABASES = { "default": { "ENGINE": "django.db.backends.sqlite3",
        "NAME": os.getenv("SQLITE_PATH", str(BASE_DIR / "db.sqlite3")),
        "OPTIONS": { "timeout": 20 } } }
```

Why this was needed. The app stored everything in SQLite — a file on local disk. In the cloud, containers are disposable; when one restarts its disk is wiped and all data is lost. This lets a `DATABASE_URL` point at a managed PostgreSQL service. **Local dev, tests and CI are unchanged** — with no `DATABASE_URL` it still uses SQLite exactly as before.

- `conn_max_age=600` — reuse a DB connection for 10 minutes.
- `conn_health_checks=True` — verify a pooled connection is alive before reuse.
- `ssl_require` — force TLS to the database in the cloud.

`requirements/base.txt` gains `dj-database-url==2.3.0` and `psycopg[binary]==3.2.13` (the driver).

9.2–9.3 · Sentry error tracking `settings.py` & `-Yatrusathi-AIchatbot/app.py`

```
SENTRY_DSN = os.getenv("SENTRY_DSN", "").strip()
if SENTRY_DSN:
    import sentry_sdk
    sentry_sdk.init(dsn=SENTRY_DSN,
        environment=os.getenv("SENTRY_ENVIRONMENT", "production"),
        traces_sample_rate=float(os.getenv("SENTRY_TRACES_SAMPLE_RATE", "0.0")),
        send_default_pii=False) # backend only
```

Sentry captures unhandled exceptions with full stack traces + request context on a dashboard. `sentry-sdk` auto-detects Django and Flask. Guarded by `if SENTRY_DSN:` — completely inert until a DSN is set. `send_default_pii=False` avoids shipping user identifiers. `sentry-sdk==2.18.0` added to both requirements files.

9.4 · Chatbot — optional Redis for scale-out `app.py`

```
socketio = SocketIO(app, cors_allowed_origins="*", async_mode="threading",
    message_queue=os.getenv("REDIS_URL") or None,    # ← added
    logger=False, engineio_logger=False)
```

What this unlocks. With `REDIS_URL` unset (default), `message_queue=None` — one process, rooms in memory, exactly as before. Set it to a Redis endpoint and Socket.IO relays room events through Redis, so multiple chatbot replicas can serve the same rooms. The code is ready; the Redis service is a documented follow-up.

9.5 · Frontend — Sentry `-yatruSathiFrontend-/src/main.tsx`

```
import * as Sentry from '@sentry/react';
const sentryDsn = import.meta.env.VITE_SENTRY_DSN;
if (sentryDsn) {
  Sentry.init({ dsn: sentryDsn,
    environment: import.meta.env.VITE_SENTRY_ENVIRONMENT ?? 'production',
    tracesSampleRate: Number(import.meta.env.VITE_SENTRY_TRACES_SAMPLE_RATE ?? 0) });
}
```

`@sentry/react` added to `package.json` + lockfile. `src/vite-env.d.ts` extended to type the new `VITE_*` variables on `import.meta.env`.

10 · Infrastructure as Code — `infra/terraform/`

Terraform describes cloud resources as declarative files. `terraform apply` makes reality match the files; `terraform destroy` removes everything. Replaces click-by-click console work with something reviewable, repeatable and deletable.

Status. Every file passes `terraform validate` and `terraform fmt`. It has **not** been applied — see §14: the available AWS account cannot create an EKS cluster.

File map

File	Creates
<code>versions.tf</code>	Provider versions (aws ~>5.70, kubernetes, helm, tls) + provider config. The k8s/helm providers authenticate by running <code>aws eks get-token</code> at apply time.
<code>variables.tf</code>	Every knob with a default (region, cluster version, node size/count, DB class, GitHub repo/branch, your admin ARN). Applies with an empty tfvars.
<code>network.tf</code>	VPC (<code>10.0.0.0/16</code>), 3 private + 3 public subnets across 3 AZs, one NAT gateway, subnet tags the load-balancer controller needs.
<code>eks.tf</code>	The EKS cluster (1.31), one managed node group, cluster add-ons (CoreDNS, kube-proxy, VPC-CNI, EBS CSI, Container Insights), and access entries — the CI role gets edit rights on <i>only</i> the two app namespaces; your ARN gets cluster-admin.
<code>namespaces.tf</code> (added)	The <code>yatusathi</code> and <code>yatusathi-staging</code> namespaces — created here because namespace creation is a cluster-level action the restricted CI role must not have.
<code>ecr.tf</code>	Three image repositories (one per service, names matching the CI push), scan-on-push, and a lifecycle policy (expire untagged after 1 day; keep only 10 tagged) so the registry doesn't grow forever.
<code>rds.tf</code>	Managed PostgreSQL 16 (<code>db.t4g.micro</code>), encrypted, 7-day backups, in private subnets, with a security group that accepts 5432 <i>only</i> from the worker nodes. Password generated by Terraform.
<code>secrets.tf</code>	Two AWS Secrets Manager secrets (<code>yatusathi/app</code> , <code>.../app-staging</code>) holding a JSON blob (DB URL, generated Django/Flask keys, <code>REPLACE_ME</code> placeholders for third-party keys). Plus an IAM role the "External Secrets Operator" pod assumes to read <i>only</i> those two secrets.
<code>iam-github-oidc.tf</code>	Registers GitHub as a trusted OIDC identity provider and creates the <code>yatusathi-github-deploy</code> role. Its trust policy accepts <i>only</i> a token from this repo on <code>main</code> . Its permissions: push to the 3 ECR repos + <code>DescribeCluster</code> on the 1 cluster. Nothing else.
<code>addons.tf</code>	Three Helm releases: AWS Load Balancer Controller (turns a k8s Ingress into a real ALB), External Secrets Operator (syncs Secrets Manager → k8s Secret every minute), metrics-server (feeds CPU/memory to the autoscalers — without it the HPAs never scale). Each gets its own narrowly-scoped IAM role (IRSA).
<code>outputs.tf</code>	Values you need after apply: <code>cluster_name</code> , <code>ecr_registry</code> , <code>github_deploy_role_arn</code> , <code>db_endpoint</code> , <code>app_secret_arns</code> .
<code>backend.tf.example</code> / <code>terraform.tfvars.example</code>	Templates for (a) storing Terraform state in an S3 bucket with locking, (b) your variable overrides. Copy, fill in, git-ignored.
<code>.terraform.lock.hcl</code>	Committed — pins exact provider versions + checksums so every <code>init</code> is identical.

Two recurring Terraform patterns

- `for_each = toset([...])` — the same block produces one resource per list item (e.g. two namespaces, two secrets, three ECR repos).
- IRSA** ("IAM Roles for Service Accounts") — a specific pod assumes a specific AWS role with no stored key. Used by all three controllers and available to the app pods.

Example — `eks.tf` access entries (the security-critical bit)

```
access_entries = merge(
  { github_ci = {                                     # the CI deploy role
    principal_arn = module.github_deploy_role.arn
    policy_associations = { edit = {
      policy_arn = ".../AmazonEKSEditPolicy"
      access_scope = { type = "namespace",
        namespaces = [var.app_namespace, "${var.app_namespace}-staging"] }
    } } },
  { for arn in var.cluster_admin_principals : "admin_${md5(arn)}" => { # you
    principal_arn = arn
    policy_associations = { admin = {
      policy_arn = ".../AmazonEKSClusterAdminPolicy", access_scope = { type = "cluster" } } } }
  }
)
```

The CI identity can edit workloads in the two app namespaces and **nothing else** — it cannot create namespaces, touch `kube-system`, or read other secrets.

11 · Kubernetes manifests — `k8s/`

Terraform builds the cluster; these files describe *what runs on it*. They use **Kustomize**: a `base/` with the shared definition, and `overlays/` that layer per-environment changes — no copy-paste.

Status. All manifests pass schema validation (`kubeconform` : 12 resources for production, 10 for staging). Not applied — no cluster exists.

11.1 · `k8s/base/kustomization.yaml`

```
namespace: yatrusathi      # stamp this namespace on every resource
resources: [configmap.yaml, externalsecret.yaml, backend.yaml, frontend.yaml, chatbot.yaml, ingress.yaml]
labels:
  - pairs: { app.kubernetes.io/part-of: yatrusathi }
    includeSelectors: false # add the label, don't touch pod selectors (they're immutable)
```

No `namespace.yaml` — Terraform creates the namespace (§10).

11.2 · `configmap.yaml` — non-secret config

A `ConfigMap` (name `yatrusathi-config`) is a bag of key/value pairs every service loads via `envFrom`: `DEBUG=False`, `ALLOWED_HOSTS`, `CORS/CSRF` origins, `DB_SSL_REQUIRE=True`, `GROQ_MODEL`, `GUNICORN_THREADS`, `REDIS_URL` (blank), `SENTRY_DSN` (blank), `SENTRY_ENVIRONMENT`. A Sentry DSN sits here rather than in Secrets because it is not a credential — it only permits *sending* events.

11.3 · externalsecret.yaml — secret sync

```
kind: SecretStore                # namespaced, so the limited CI role can create it
metadata: { name: aws-secretsmanager }
spec: { provider: { aws: { service: SecretsManager, region: ap-south-1,
    auth: { jwt: { serviceAccountRef: { name: external-secrets, namespace: external-secrets } } } } } }
---
kind: ExternalSecret
metadata: { name: yatrusathi-app-secrets }
spec:
  refreshInterval: 1m
  secretStoreRef: { name: aws-secretsmanager, kind: SecretStore }
  target: { name: yatrusathi-app-secrets, creationPolicy: Owner }
  data:
    # map 6 keys out of the AWS secret
    - { secretKey: DATABASE_URL,      remoteRef: { key: yatrusathi/app, property: DATABASE_URL } }
    - { secretKey: DJANGO_SECRET_KEY, ... } ... CHATBOT_SECRET_KEY, GROQ_API_KEY, RESEND_API_KEY,
      DJANGO_ADMIN_PASSWORD
```

The `ExternalSecret` is a request: "read these keys from AWS secret `yatrusathi/app` , produce a Kubernetes Secret `yatrusathi-app-secrets` , refresh every minute". The operator does it using its IRSA role; pods just mount the resulting native Secret and never call AWS.

11.4 · backend.yaml — Deployment + Service + HPA

```
kind: Deployment (replicas: 2)
  initContainers:
    - name: migrate                # runs to completion BEFORE the app container
      image: backend
      command: ["python", "manage.py", "migrate", "--no-input"]
      env: SECRET_KEY+DJANGO_SECRET_KEY, DATABASE_URL (from the Secret)
  containers:
    - name: backend image: backend port 8000
      envFrom: [configMapRef: yatrusathi-config]
      env: SECRET_KEY, DATABASE_URL, RESEND_API_KEY, ADMIN_PASSWORD (from the Secret)
      readinessProbe: GET /api/ (X-Forwarded-Proto: https) # no traffic until this passes
      livenessProbe: GET /api/ # restart if this starts failing
      resources: requests {cpu 100m, mem 256Mi} limits {cpu 1, mem 512Mi}
---
kind: Service (name backend, port 8000) # stable internal name, load-balances across pods
---
kind: HorizontalPodAutoscaler (backend, 2-6 replicas, target 70% CPU)
```

Object	Role
Deployment	Keeps N identical pods running; rolling-replaces them on a new image.
initContainer <code>migrate</code>	DB migrations run to completion before any app container starts.
readinessProbe / livenessProbe	Gate traffic until ready; restart if unhealthy.
<code>X-Forwarded-Proto: https</code> header	Mimics the load balancer so Django's HTTPS redirect doesn't loop the health check.
requests / limits	"requests" is reserved by the scheduler; "limits" is the hard cap (CPU throttled, memory over-limit = killed).
Service	A stable virtual IP + DNS name that spreads traffic across the pods despite churn.
HPA	Adds/removes pods to hold average CPU near target. Needs metrics-server.

Known trade-off. `migrate` as an initContainer runs on every pod start. Migrations are idempotent so it's safe, but two pods starting together could race. For zero-downtime, promote it to a single pre-deploy Job. Documented, not yet changed.

11.5 · `frontend.yaml`

Same shape, simpler — no secrets (the bundle is static), no initContainer. `replicas: 2`, HPA 2→5 at 75% CPU, tiny requests (25m CPU / 32Mi) because nginx serving static files is cheap. Probes hit `GET /`.

11.6 · `chatbot.yaml`

```
kind: Deployment
  replicas: 1                # deliberate — in-memory rooms ($5.3)
  strategy: { type: Recreate } # stop the old pod before starting the new
  containers: [ chatbot, port 5005, env SECRET_KEY=CHATBOT_SECRET_KEY + GROQ_API_KEY, probes GET /health ]
---
kind: Service
  sessionAffinity: ClientIP    # pin each client to one pod for its socket's life
  ports: [ 5005 ]
```

No HPA. The file's comment tells the next engineer how to scale it: set `REDIS_URL` in the ConfigMap, raise `replicas`.

11.7 · `ingress.yaml` — one ALB, path routing

```
annotations:
  alb.ingress.kubernetes.io/scheme: internet-facing
  alb.ingress.kubernetes.io/target-type: ip
  alb.ingress.kubernetes.io/listen-ports: '[{"HTTP":80}]'
  alb.ingress.kubernetes.io/load-balancer-attributes: idle_timeout.timeout_seconds=3600
  alb.ingress.kubernetes.io/group.name: yatusathi
rules: [ paths, IN ORDER:
  /socket.io → chatbot:5005
  /api/chat → chatbot:5005
  /api/session → chatbot:5005
  /api → backend:8000
  /admin → backend:8000
  /static → backend:8000
  / → frontend:80 ]
```

Why the order matters. The AWS Load Balancer Controller evaluates rules top-to-bottom. The chatbot's specific paths must precede the backend's catch-all `/api`, and everything precedes the frontend's `/`. One hostname, one ALB, three services — no extra domains. `idle_timeout 3600` keeps the chatbot's SSE/WebSocket connections open.

11.8 · Overlays

	production	staging (added)
Namespace	yatrusathi	yatrusathi-staging
Images	point <code>image:</code> names at the real ECR registry; CI rewrites the tag to the commit SHA each deploy	same
Patches	narrow CORS/CSRF origin to the ALB DNS (stub)	read secrets from <code>yatrusathi/app-staging</code> ; <code>SENTRY_ENVIRONMENT=staging</code> ; its own ALB (<code>group.name</code>); <code>replicas: 1</code> ; delete both HPAs

A Kustomize patch is a small JSON-Patch / strategic-merge snippet — e.g. `{op: replace, path: /spec/replicas, value: 1}`. Staging = the same base with five small edits.

12 · The deploy pipeline

12.1 · Two environments, one script

	deploy-staging	deploy-production
Trigger	automatic on every <code>main</code> push (after <code>push-ecr</code>)	manual — "Run workflow" (<code>workflow_dispatch</code>)
Overlay / namespace	<code>k8s/overlays/staging</code> / <code>yatrusathi-staging</code>	<code>k8s/overlays/production</code> / <code>yatrusathi</code>
<code>environment:</code>	<code>staging</code>	<code>production</code> — attach required reviewers in GitHub to gate it

Both install `kubectl` + `kustomize`, authenticate via OIDC, then run the shared `.github/scripts/deploy-eks.sh`. Everything they differ on is passed as environment variables.

12.2 · `.github/scripts/deploy-eks.sh`

```
set -euo pipefail
: "${AWS_REGION:?}" "${EKS_CLUSTER_NAME:?}" "${OVERLAY:?}" "${NAMESPACE:?}" "${ENV_NAME:?}" "${SHA:?}"

ACCOUNT_ID="$(aws sts get-caller-identity --query Account --output text)"
REGISTRY="${ACCOUNT_ID}.dkr.ecr.${AWS_REGION}.amazonaws.com"
aws eks update-kubeconfig --name "$EKS_CLUSTER_NAME" --region "$AWS_REGION"

frontend_tag="$SHA"
if [[ -n "${APP_HOST:-}" ]]; then                                # rebuild the frontend for THIS env's URLs
    frontend_tag="$SHA-${ENV_NAME}"
    aws ecr get-login-password ... | docker login ... "$REGISTRY"
    docker build ./-yatruSathiFrontend- \
        --build-arg "VITE_API_BASE_URL=https://${APP_HOST}/api/" \
        --build-arg "VITE_CHATBOT_URL=https://${APP_HOST}" \
        --build-arg "VITE_SENTRY_DSN=${VITE_SENTRY_DSN:-}" \
        --build-arg "VITE_SENTRY_ENVIRONMENT=${ENV_NAME}" \
        --build-arg "VITE_SUPABASE_URL=${VITE_SUPABASE_URL:-}" \
        --build-arg "VITE_SUPABASE_ANON_KEY=${VITE_SUPABASE_ANON_KEY:-}" \
        -t "${REGISTRY}/yatru-sathi-frontend:${frontend_tag}"
    docker push "${REGISTRY}/yatru-sathi-frontend:${frontend_tag}"
else
    echo "::warning::APP_HOST not set for ${ENV_NAME} - frontend will call localhost."
fi

pushd "$OVERLAY" >/dev/null
kustomize edit set image \
    "backend=${REGISTRY}/yatru-sathi-backend:${SHA}" \
    "frontend=${REGISTRY}/yatru-sathi-frontend:${frontend_tag}" \
    "chatbot=${REGISTRY}/yatru-sathi-chatbot:${SHA}"
popd >/dev/null

kubectl apply -k "$OVERLAY"
for d in backend frontend chatbot; do
    kubectl -n "$NAMESPACE" rollout status "deployment/${d}" --timeout=180s
done
```

Line	What & why
<code>: "\${VAR:?}"</code>	Abort with a clear message if a required variable is missing.
<code>aws eks update-kubeconfig</code>	Write a kubeconfig so <code>kubectl</code> can reach the cluster.
the <code>APP_HOST</code> block	Vite bakes URLs in at build time, so the frontend image is environment-specific: rebuild it with this env's public host, tag <code><sha>-staging</code> / <code>-production</code> . Backend/chatbot images are host-agnostic (built once by <code>push-ecr</code>).
<code>kustomize edit set image ...</code>	Rewrite the manifests to reference the images just pushed.
<code>kubectl apply -k</code>	Send the rendered manifests to the cluster.
<code>kubectl rollout status ... --timeout=180s</code>	Wait for every Deployment to report its new pods healthy; fail the job otherwise — this is what turns a bad deploy into a red pipeline.

Why a script, not inline YAML. Both deploy jobs run the identical sequence — one place to fix a bug, and the logic can be read and tested on its own.

13 · Legacy / superseded deploy configs

Still in the tree (mostly under `-yatrubackend-/`), kept for history and because the backend's `.dockerignore` excludes them from the image. **None are used by the current pipeline.**

File	What it was for	Superseded by
<code>-yatrubackend-/render.yaml</code>	Render "Blueprint": a free-tier Python web service, build <code>scripts/build.sh</code> , start <code>scripts/start.sh</code> , auto-deploy on push. Env: <code>PYTHON_VERSION=3.11.7</code> , <code>DEBUG=False</code> , generated <code>SECRET_KEY</code> , <code>ALLOWED_HOSTS=.onrender.com</code> .	the container image + <code>docker-entrypoint.sh</code> ; RDS instead of Render's ephemeral disk
<code>-yatrubackend-/Procfile</code>	<code>web: scripts/start.sh</code> — the process Render/Heroku runs.	the image's <code>ENTRYPOINT</code>
<code>-yatrubackend-/runtime.txt</code>	<code>python-3.11.7</code> — pins the Python version for buildpack hosts.	<code>FROM python:3.13-slim</code> in the Dockerfile
<code>-yatrubackend-/scripts/build.sh</code>	<code>pip install</code> → <code>collectstatic</code> → <code>migrate</code> (Render build step).	the Dockerfile builder stage + <code>entrypoint</code>
<code>-yatrubackend-/scripts/start.sh</code>	<code>migrate</code> → <code>seed</code> → <code>gunicorn</code> (Render start step).	<code>docker-entrypoint.sh</code> (same steps, env-driven)
<code>-yatrubackend-/scripts/setup.sh</code>	Local dev bootstrap: create venv, install, migrate, make media dirs.	<code>docker compose up</code>
<code>-yatrubackend-/scripts/deploy_setup.sh</code>	One-off "git init + push + Render instructions" helper. Prints hard-coded DB credentials (<code>YatruSathi@123</code> , a Supabase host) — flagged in <code>docs/secret-rotation.md</code> .	the CI/CD pipeline; secrets now in Secrets Manager
<code>-yatruSathiFrontend-/.github/workflows/deploy.yml</code>	Vendored from the old frontend repo: build and push <code>dist/</code> to Cloudflare Pages on every push (needs <code>CLOUDFLARE_API_TOKEN</code> secrets). Inert here — the workflow name/secrets don't exist in this repo.	the frontend container image + <code>push-ecr</code> + the deploy jobs
<code>-yatrubackend-/scripts/seed_db.py</code>	Still used — idempotent sample-data loader, called by <code>docker-entrypoint.sh</code> (non-fatal).	—

These can be deleted once the container path is proven in production; they are left now as a reference and a fallback.

14 · Current status & the AWS-account constraint

What is done and verified

Area	State
Monorepo consolidation + env-var strategy	Done (committed)
Backend & frontend Dockerfiles + compose (backend/frontend)	Done (pre-existing, in use)
Chatbot containerised, in compose & CI	Done, tested — image builds, runs, health check passes
Backend Postgres support	Done, tested — works with and without DATABASE_URL
CI: build/test/lint, Trivy deps + images	Done (committed)
CI: OIDC push-to-ECR + deploy jobs	Done (committed), not yet run against real infra
Dependabot config, CodeQL, gitleaks, hadolint, caching	Done — workflow YAML validated (uncommitted branch)
Sentry in all three services; chatbot Redis-ready	Done — build/typecheck/import pass; inert until configured
Terraform (VPC, EKS, RDS, ECR, IAM, Secrets, add-ons, staging)	Written & validated (validate + fmt); not applied
Kubernetes manifests (base + 2 overlays)	Written & validated (schema-checked); not applied
Docs: environment.md , deployment-eks.md , secret-rotation.md , this report	Done

The blocker: this AWS account cannot run EKS

The available account is a restricted **bootcamp / student** account. A permission check (`aws iam simulate-principal-policy`) shows:

Needed by the EKS design	This account
<code>eks:*</code> (create a cluster)	denied
<code>ecs:*</code>	denied
<code>iam:CreateOpenIDConnectProvider</code> (GitHub→AWS OIDC)	denied
<code>secretsmanager:*</code>	denied
EC2 / VPC / ELB / RDS / ECR / IAM roles / SSM / KMS	allowed — region locked to <code>ap-south-1</code>

So the EKS Terraform and the `k8s/` manifests **cannot be deployed on this account**. They stay as valid, reusable work for any account that permits EKS.

The path that fits: EC2 + docker compose

GitHub push → CI (build / test / scan) → push images to ECR
→ `aws ssm send-command` → EC2: `docker compose pull && up -d`
EC2 (t3.medium) — docker compose — frontend / backend / chatbot
├ RDS PostgreSQL (db.t4g.micro)
├ secrets in SSM Parameter Store (replaces Secrets Manager)
└ Application Load Balancer in front (HTTPS once a domain exists)

Carries over unchanged	Gets replaced
All 3 Dockerfiles + <code>docker-compose.yml</code> (becomes the real runtime)	<code>infra/terraform/</code> → smaller: VPC + 1 EC2 + RDS + ECR + SSM + ALB + a CI IAM user
Backend <code>DATABASE_URL</code> , Sentry, Redis wiring	<code>k8s/</code> → removed (no Kubernetes)
Every CI job except deploy (build, test, Trivy, CodeQL, gitleaks, hadolint, Dependabot)	OIDC → an IAM user access key in GitHub Secrets (the repo's original approach)
<code>.env.example</code> inventory; <code>secret-rotation.md</code> (retargeted at SSM)	deploy step → <code>aws ssm send-command</code> running <code>docker compose pull && up -d</code>

Rough cost of the EC2 path left running: ~\$45–70/month (instance + RDS + ALB + NAT), less if the ALB/NAT are dropped for a demo.

Decisions still owned by a human

- **Rotate the previously-leaked credentials** (OpenAI, Google, Supabase, Gmail) — needs those third-party accounts. Steps in `docs/secret-rotation.md` ; `gitleaks` blocks re-introduction.
- **Approve the EC2 pivot** so the infrastructure code can be rebuilt for what this account permits.
- **Commit & push** the current branch and merge to `main` — CI, Dependabot and CodeQL only fully activate there.